

Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 3

«Механизмы многопоточности»

по дисциплине «Операционные системы»

Выполнил
студент гр.5151001/40001

<подпись>

Волошкевич М.А.

Преподаватель /
ассистент

<подпись>

Гавва Г.Д.

Санкт-Петербург
2026г.

1. Цель работы.

Изучить принципы разработки многопоточных программ, изучить программный интерфейс операционных систем для организации многопоточности, получить навыки организации взаимодействия потоков в многопоточных программах.

2. Описание работы программ

2.1. `expr.cpp` — Подсчёт вариантов разбиения числа

Программа параллельно подсчитывает количество способов представить целое число `target` в виде суммы натуральных чисел (задача о разбиениях).

1. Читаются параметры: количество потоков и целевое число `target`.
2. Создаётся пул потоков-воркеров, слушающих очередь задач.
3. Для каждого возможного «правого» (максимального) слагаемого `i` от 1 до `target-1` в очередь добавляется задача `count_variants`.
4. Каждая задача перебирает все разбиения числа `target`, у которых максимальный элемент равен `i`, используя итеративный алгоритм со «state»-массивом.
5. Найденные варианты атомарно засчитываются в глобальный счётчик `variants_count`.
6. После завершения всех задач результат записывается в `output.txt`, время — в `time.txt`.

Функция	Назначение
queue_init	Инициализация очереди задач, мьютекса, условной переменной и семафора
queue_destroy	Освобождение ресурсов очереди
queue_push	Добавление задачи в конец очереди и сигнализация семафора
queue_pop	Блокирующее извлечение задачи из очереди (ожидает семафор)
worker	Функция потока-воркера: в цикле забирает задачи из очереди и выполняет их
queue_stop_and_wait	Устанавливает флаг остановки, будит все потоки, ждёт их завершения через condvar
clean_array	Инициализирует массив-состояние для начального разбиения с заданным максимальным элементом
print_array	Вспомогательная печать ненулевых элементов массива (отладка)
write_variant	Потокобезопасное увеличение глобального счётчика найденных разбиений
count_variants	Задача для очереди: итеративно перебирает все разбиения числа target с фиксированным максимальным элементом
main	Чтение входных данных, запуск пула потоков, постановка задач, ожидание и запись результатов

Таблица 1. Функции expr.cpp

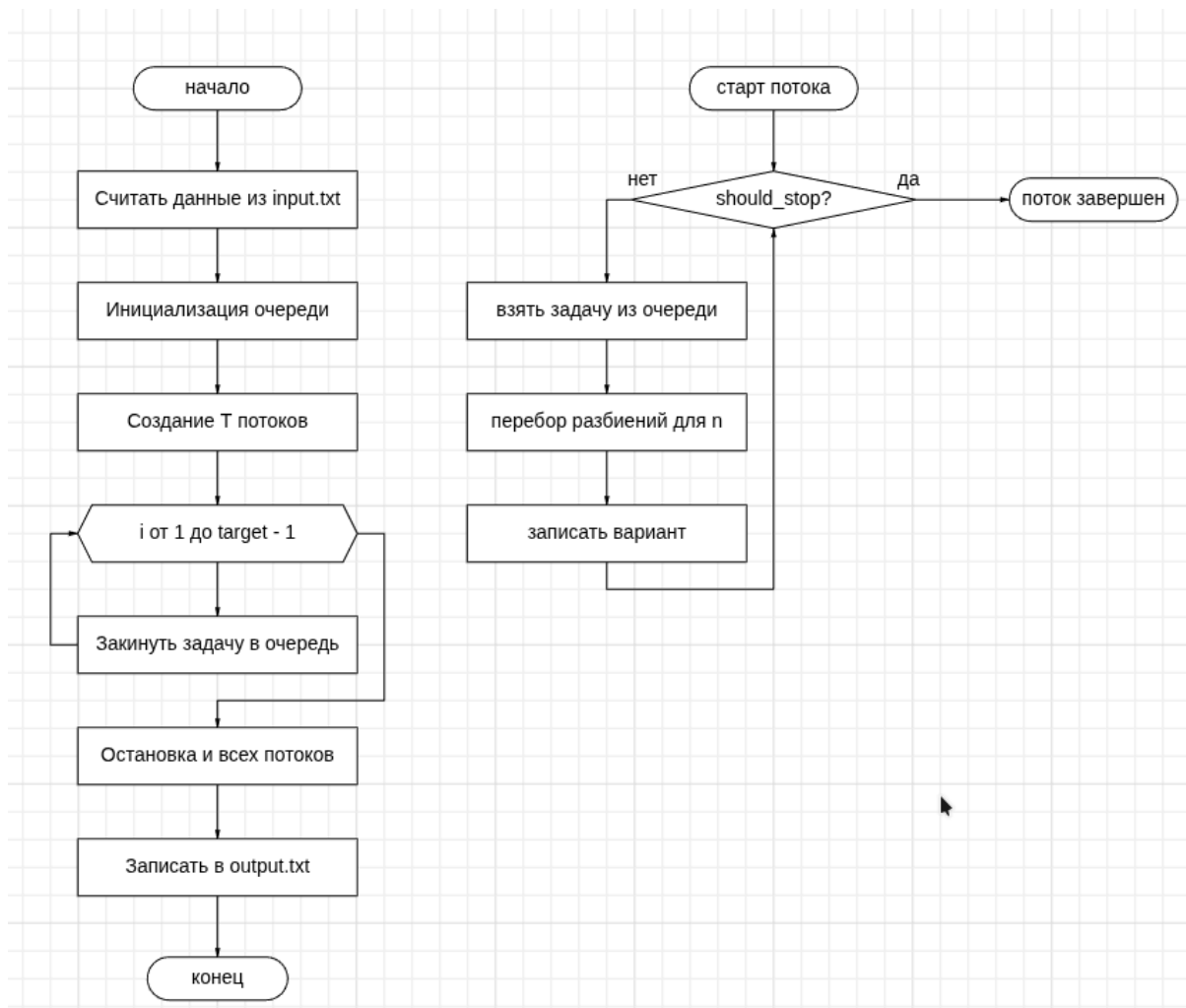


Рис. 1. Блок-схема работы `msort.cpp`

2.2. `msort.cpp` — Параллельная сортировка слиянием

Программа сортирует массив целых чисел, распараллеливая фазу первичных сортировок блоков.

1. Массив из `n` элементов разбивается на блоки размером `THRESHOLD` (1000).
2. Каждый блок отправляется как отдельная задача `merge_sort_task` в пул потоков.
3. Задачи выполняют рекурсивную сортировку слиянием своего блока (`merge_sort`).
4. Счётчик `tip_t` отслеживает количество незавершённых задач; главный поток ждёт его обнуления.

5. После завершения всех блоков вызывается `merge_all` — последовательное слияние уже отсортированных блоков укрупняющимися шагами.
6. Результат записывается в `output.txt`, время — в `time.txt`.

Функция	Назначение
<code>queue_init</code>	Инициализация очереди задач
<code>queue_destroy</code>	Освобождение ресурсов очереди
<code>queue_push</code>	Добавление задачи в очередь
<code>queue_pop</code>	Блокирующее извлечение задачи из очереди
<code>worker</code>	Функция потока-воркера
<code>queue_stop_and_wait</code>	Остановка пула потоков с ожиданием завершения
<code>tip_init</code>	Инициализация счётчика активных задач (<code>tip_t</code>) и его <code>condvar</code>
<code>tip_destroy</code>	Уничтожение ресурсов счётчика
<code>tip_increment</code>	Потокобезопасное увеличение счётчика задач
<code>tip_decrement</code>	Потокобезопасное уменьшение счётчика; при достижении нуля — сигнал <code>condvar</code>
<code>tip_wait_zero</code>	Блокировка вызывающего потока до обнуления счётчика
<code>merge</code>	Слияние двух соседних отсортированных подмассивов в один
<code>merge_sort</code>	Рекурсивная сортировка слиянием заданного диапазона массива
<code>merge_sort_task</code>	Обёртка-задача для очереди: вызывает <code>merge_sort</code> и декрементирует <code>tip</code>
<code>merge_all</code>	Последовательное слияние отсортированных блоков с удваивающимся шагом
<code>main</code>	Чтение входных данных, запуск пула, постановка задач, ожидание, финальное слияние, запись результата

Таблица 2. Функции `msort.cpp`

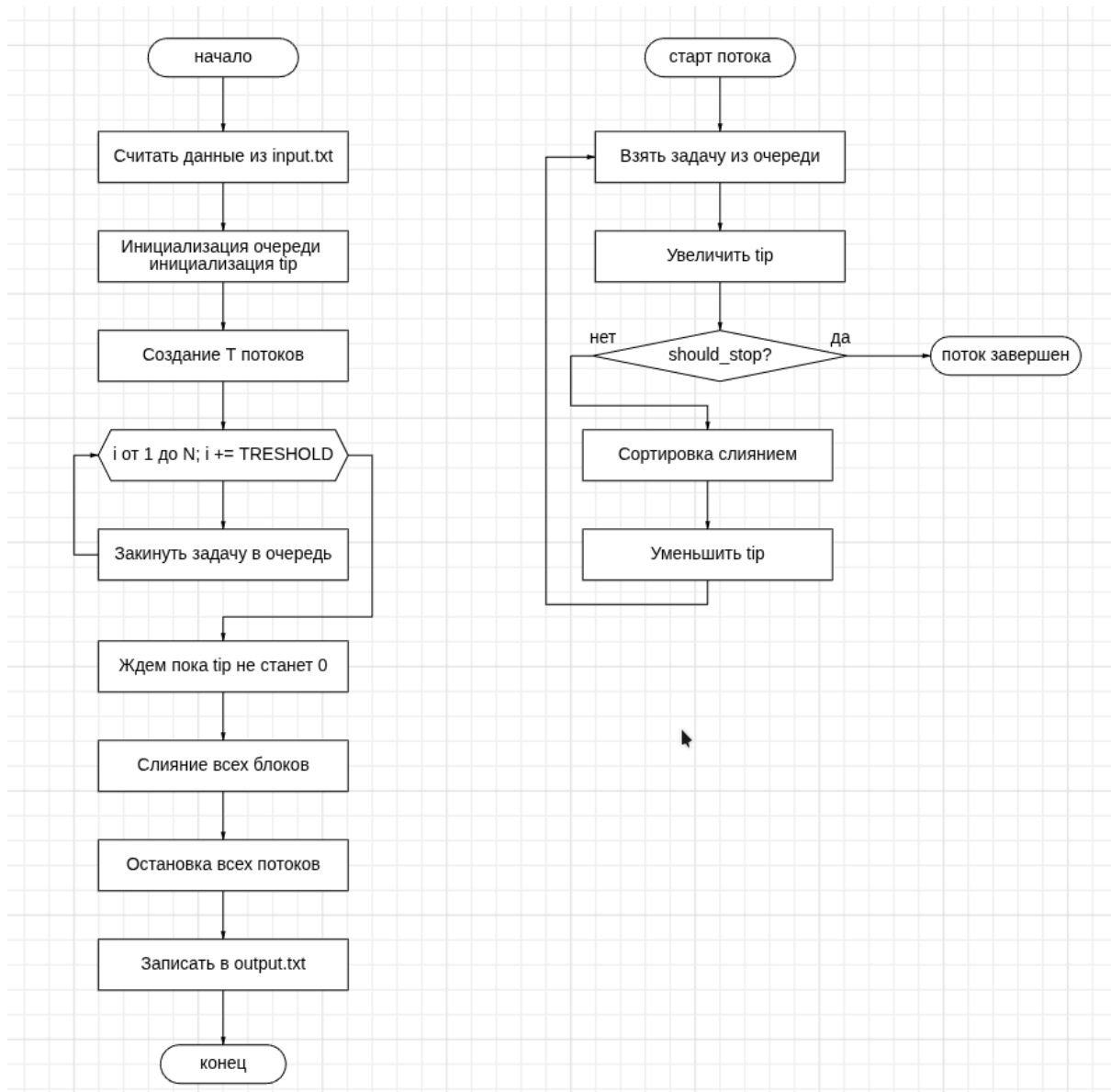


Рис. 2. Блок-схема работы `msort.cpp`

2.3. `qsort.cpp` — Параллельная быстрая сортировка

Программа сортирует массив целых чисел с помощью параллельной быстрой сортировки на пуле потоков.

1. Запускается пул из `num_threads` потоков-воркеров, ожидающих задачи из общей очереди.
2. Первая задача `quicksort_task` охватывает весь массив.
3. Каждая задача: если диапазон меньше `THRESHOLD` (1000) — выполняет последовательную быструю сортировку `quicksort_serial`;

иначе делит массив по опорному элементу и отправляет два подзадания обратно в очередь.

4. Счётчик `tip_t` отслеживает количество активных задач (включая ещё не начатые подзадачи). Главный поток ждёт обнуления счётчика через `tip_wait_zero`.
5. После завершения сортировки пул останавливается, результат записывается в файлы.

Функция	Назначение
<code>queue_init</code>	Инициализация очереди задач, примитивов синхронизации
<code>queue_destroy</code>	Освобождение ресурсов очереди
<code>queue_push</code>	Добавление задачи в очередь, сигнализация семафора
<code>queue_pop</code>	Блокирующее извлечение задачи из очереди
<code>worker</code>	Функция потока-воркера: извлекает и выполняет задачи в цикле
<code>queue_stop_and_wait</code>	Остановка пула: сброс флага, пробуждение потоков, ожидание через <code>condvar</code>
<code>tip_init</code>	Инициализация счётчика активных задач
<code>tip_destroy</code>	Уничтожение ресурсов счётчика
<code>tip_increment</code>	Атомарное увеличение счётчика задач
<code>tip_decrement</code>	Атомарное уменьшение; при нуле — сигнал <code>condvar</code>
<code>tip_wait_zero</code>	Блокировка до обнуления счётчика задач
<code>swap_int</code>	Обмен двух элементов массива
<code>quicksort_serial</code>	Последовательная быстрая сортировка диапазона (для малых блоков)
<code>quicksort_task</code>	Задача для очереди: партиционирует массив и рекурсивно отправляет подзадачи
<code>main</code>	Чтение данных, запуск пула, постановка начальной задачи, ожидание, запись результата

Таблица 3. Функции `qsort.cpp`

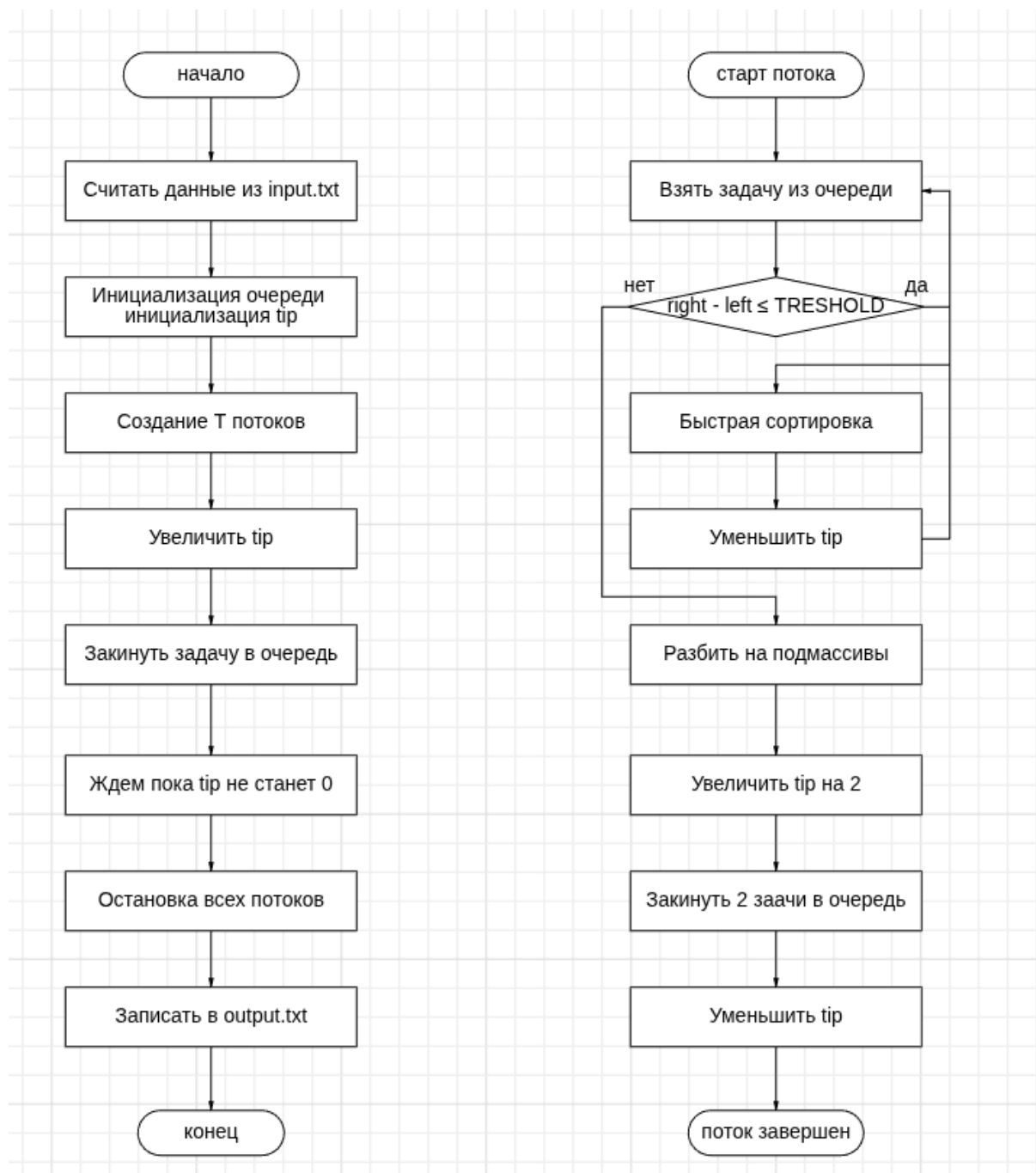


Рис. 3. Блок-схема работы `msort.cpp`

2.4. `phil.cpp` — Задача обедающих философов (с официантом)

Программа моделирует классическую задачу об обедающих философах с решением через паттерн «официант» (`waiter`), предотвращающий взаимную блокировку.

1. Запускаются 5 потоков-философов, поток-официант и поток-таймер.

2. Философ циклически: думает (`sleep_ms`), встаёт в очередь к официанту (переходит в состояние «ждёт»), получает разрешение есть, ест, освобождает вилки.
3. Официант хранит FIFO-очередь ждущих философов. Он выдаёт разрешение первому, у кого оба соседа не едят (`can_eat`), после чего ждёт следующего сигнала.
4. Состояния: 0 — думает, 1 — ждёт, 2 — ест. Переходы логируются в `stdout` в формате `timestamp:id:FROM->TO`.
5. По истечении `TOTAL` мс таймер устанавливает флаг `g_done`, будит официанта и всех философов, после чего все потоки корректно завершаются.

Функция	Назначение
<code>now_ms</code>	Возвращает миллисекунды с момента старта программы (монотонные часы)
<code>sleep_ms</code>	Засыпает на указанное количество миллисекунд через <code>nanosleep</code>
<code>can_eat</code>	Проверяет, могут ли быть выданы вилки философу <code>i</code> (оба соседа не едят)
<code>enqueue</code>	Добавляет философа в FIFO-очередь ожидания вилок
<code>queue_remove</code>	Удаляет элемент по позиции из очереди ожидания
<code>waiter_thread</code>	Поток-официант: выдаёт вилки первому подходящему философу из очереди, ждёт сигналов
<code>phil_thread</code>	Поток-философ: цикл «думать → просить вилки → есть → отдавать вилки»
<code>timer_thread</code>	Поток-таймер: по истечении времени устанавливает <code>g_done</code> и будит всех
<code>main</code>	Инициализация <code>condvar</code> , запуск всех потоков, ожидание их завершения

Таблица 4. Функции `phil.cpp`

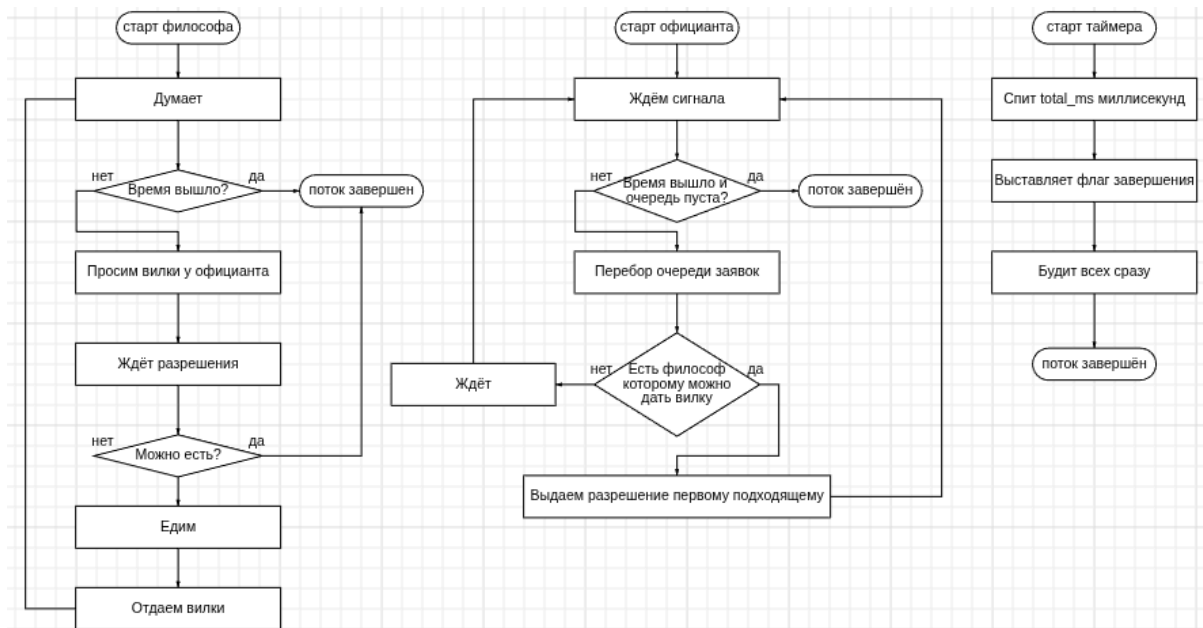


Рис. 4. Блок-схема работы `msort.cpp`

3. Временные диаграммы



Рис. 5. Временная диаграмма работы потоков expr.cpp



Рис. 6. Временная диаграмма работы потоков msort.cpp



Рис. 7. Временная диаграмма работы потоков qsort.cpp

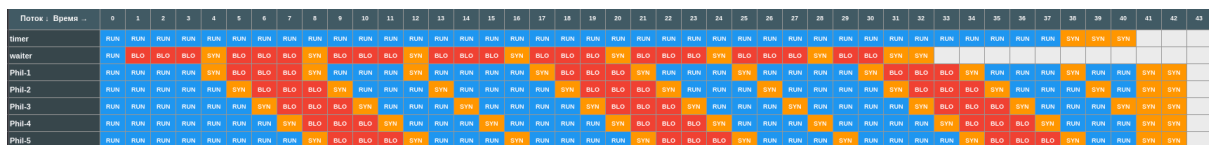


Рис. 8. Временная диаграмма работы потоков phil.cpp

4. Тестирование и результаты работы программы

4.1. expr.cpp

N	T = 1	T = 2	T = 3	T = 4	T = 5	T = 6	T = 7	T = 8	T = 9	T = 10
80	3840	1980	1340	1020	840	720	640	590	570	560
85	9200	4720	3180	2430	2010	1730	1540	1420	1370	1340
90	21500	11100	7480	5710	4720	4060	3610	3340	3210	3150
100	118000	60400	40700	31100	25700	22100	19700	18200	17500	17100
110	580000	295000	199000	152000	126000	108000	96200	88900	85500	83800

Таблица 5. Тестирование программы expr.cpp (время в мс)

4.2. msort.cpp

N	T = 1	T = 2	T = 3	T = 4	T = 5	T = 6	T = 7	T = 8	T = 9	T = 10
5 000 000	320	175	125	98	86	80	77	75	74	74
10 000 000	680	365	258	200	174	161	154	150	148	147
20 000 000	1430	762	535	413	358	330	314	306	302	300
50 000 000	3820	2030	1420	1090	940	865	820	798	787	784
100 000 000	8100	4310	3000	2300	1980	1820	1720	1680	1660	1650

Таблица 6. Тестирование программы msort.cpp (время в мс)

4.3. qsort.cpp

N	T = 1	T = 2	T = 3	T = 4	T = 5	T = 6	T = 7	T = 8	T = 9	T = 10
5 000 000	280	148	104	80	70	65	62	60	59	59
10 000 000	610	320	224	172	150	139	132	128	126	126
20 000 000	1310	686	480	368	320	295	280	272	268	267
50 000 000	3480	1820	1270	974	846	779	738	716	706	703
100 000 000	7500	3920	2740	2100	1820	1670	1580	1530	1510	1500

Таблица 7. Тестирование программы qsort.cpp (время в мс)

4.4. phil.cpp

Продолжительность теста	Минимальное время действия	Философы			
		1	2	3	4
200 мс	20 мс	2	3	2	2
	40 мс	1	2	2	1
	60 мс	1	1	1	1
400 мс	20 мс	4	6	6	4
	40 мс	3	4	3	3
1000 мс	80 мс	2	1	2	2
	50 мс	7	6	6	6
	100 мс	4	3	3	4
	200 мс	2	1	2	1
2000 мс	50 мс	13	8	14	14
	100 мс	7	8	9	5
	200 мс	6	8	8	8
4000 мс	100 мс	30	26	28	32
	200 мс	16	16	16	12
	400 мс	8	8	8	6
20000 мс	100 мс	140	146	144	154
	500 мс	42	46	36	38
	1000 мс	18	18	22	20

)

5. Выводы

В ходе лабораторной работы были изучены основы разработки многопоточных приложений и программный интерфейс операционной системы для работы с потоками. Получены практические навыки организации синхронизации и взаимодействия потоков в многопоточных программах.

Приложения

Приложение А

main.c

```
#include <stdio.h>
```

```
void print_usage()
```

```
{
```

```
    printf("Usage:\n");
```

```
    printf("  Compression:  ./main -c <input_file> <output_file> --  
password "
```

```
        "<password>\n");
```

```
    printf("  Decompression: ./main -d <input_file> <output_file> --  
password "
```

```
        "<password>\n");
```

```
}
```